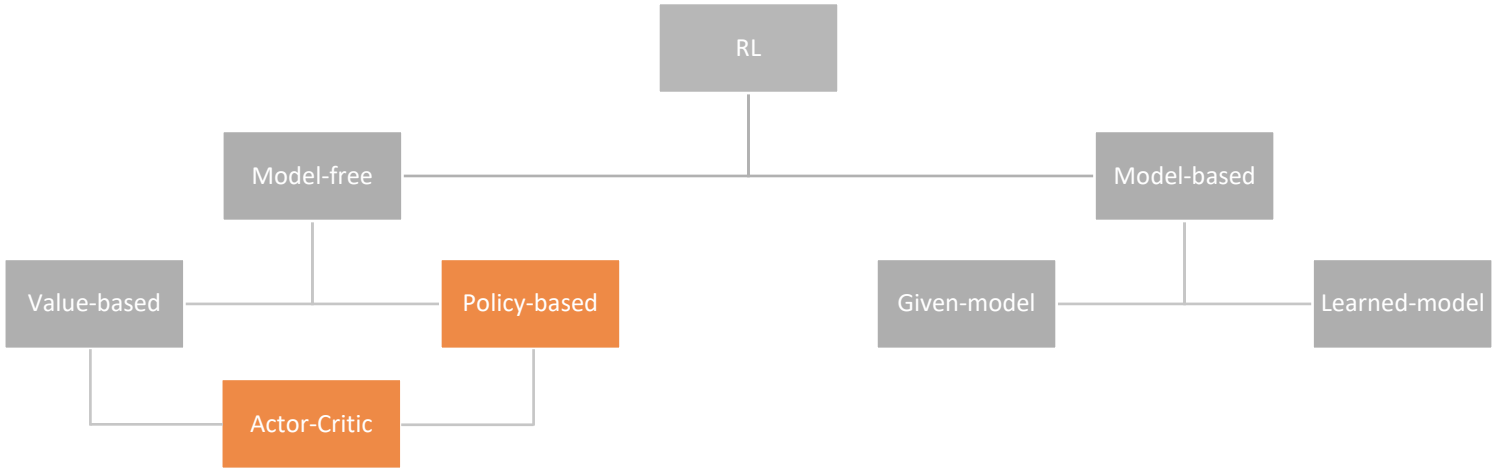
Robot Learning

Exercise 3 – Policy Gradient RL

Topics of today lab

- Policy Gradient algorithms
- REINFORCE
- Actor Critic methods (PPO, SAC)
- Stable-Baselines3

Bird's-Eye view of Reinforcement Learning algorithms



What do we optimize?

- **Value-based:** the policy can then be derived from the **learned value function**.

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

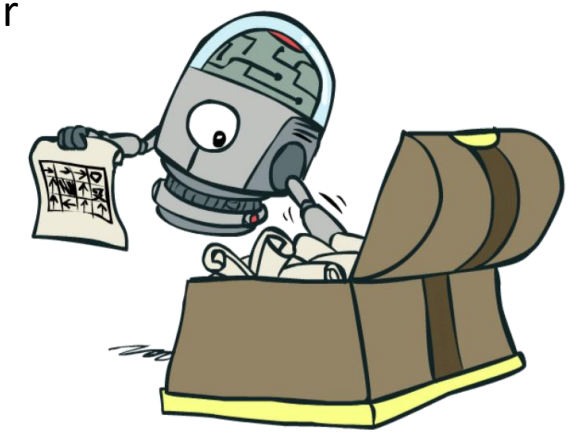
- **Policy-based: directly learn the policy** without explicitly learning the value function.

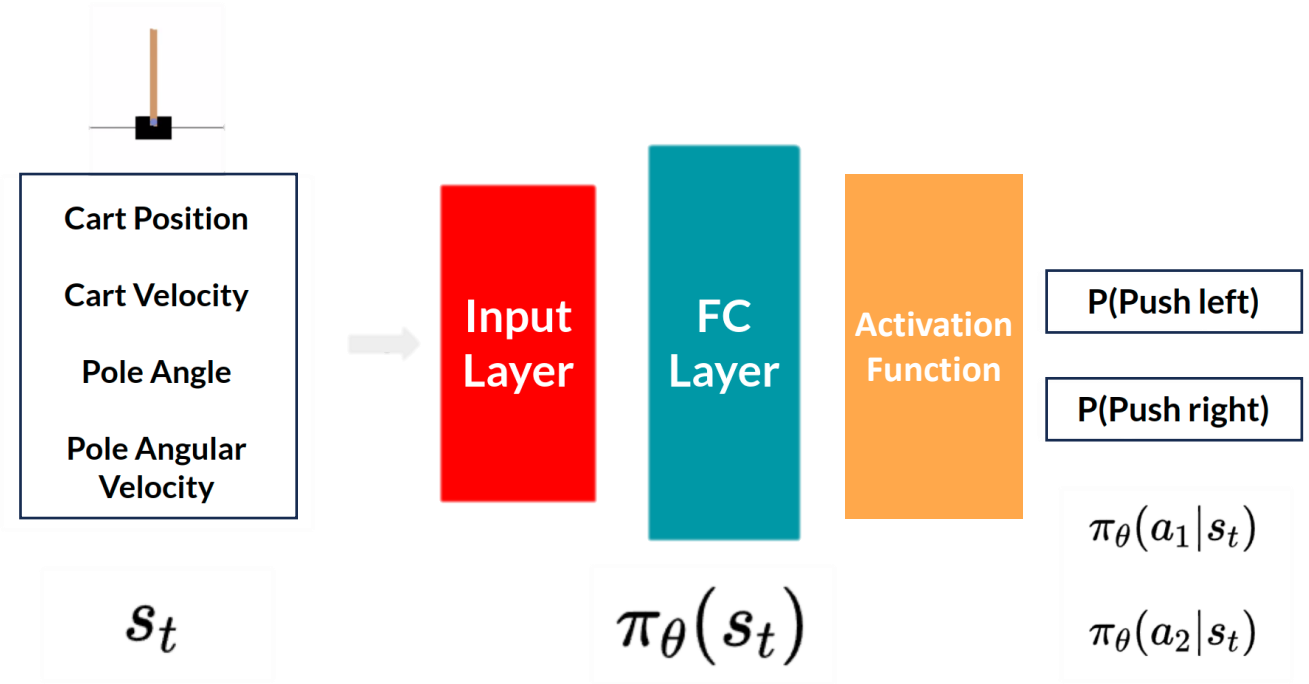
$$\pi_\theta(s) = \mathbb{P}[A|s;\theta]$$

- **Actor-Critic:** learn both the **value function** and the **policy**.

Policy-based methods

- The agent does not **select the best action** based on state-action values but rather **from a probability distribution of actions given a state**, based on the current **parametrized stochastic policy** $\pi(a \mid s, \theta)$

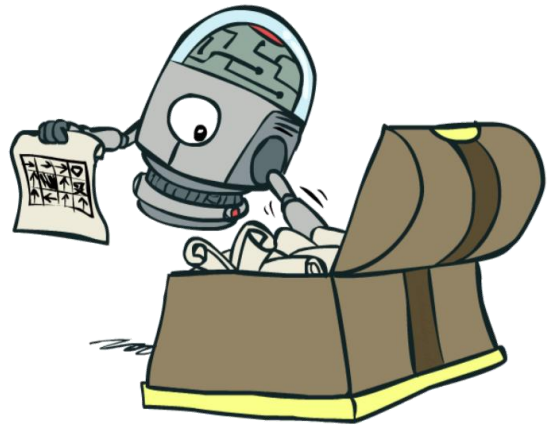




Policy Gradient algorithms

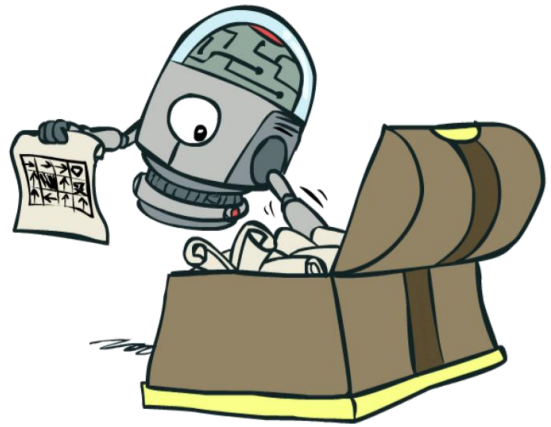
- The agent does not **select the best action** based on state-action values but rather **from a probability distribution of actions given a state**, based on the current **parametrized stochastic policy** $\boldsymbol{\pi(a \mid s, \theta)}$
- Optimal Policy: **maximize** the expected return using the **objective function**

$$J(\pi_\theta) = E_{\pi_\theta}[\sum_{t=0}^{T-1} R(s_t, a_t)] = E_{\pi_\theta}[R(\tau)]$$



Policy Gradient algorithms

- The agent does not **select the best action** based on state-action values but rather **from a probability distribution of actions given a state**, based on the current **parametrized stochastic policy** $\boldsymbol{\pi(a \mid s, \theta)}$



- Optimal Policy: **maximize** the expected return using the **objective function**

$$J(\pi_\theta) = E_{\pi_\theta}[\sum_{t=0}^{T-1} R(s_t, a_t)] = E_{\pi_\theta}[R(\tau)]$$

  - Use of **gradient ascent** to adjust the policy parameters to find the optimal policy:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \nabla J(\boldsymbol{\theta}_t)$$

  - **Policy Gradient theorem**: $\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)]$

REINFORCE

- Naive algorithm in the family of policy gradient algorithms
- It relies on an estimated return by **Monte-Carlo methods** using **episode sampling** to update the policy parameter θ and estimate the gradient $\hat{g} \approx \nabla_\theta J(\theta)$

$$\nabla_\theta J(\theta) \approx \hat{g} = \sum_{t=0} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)$$

Estimation of the gradient (given we use only one trajectory to estimate the gradient)

Probability of the agent to select action at from state st given our policy

Cumulative return

Direction of the steepest increase of the (log) probability of selecting action at from state st

- Update rule: $\theta_{t+1} = \theta_t + \alpha G_t \nabla \ln \pi(A_t|S_t)$

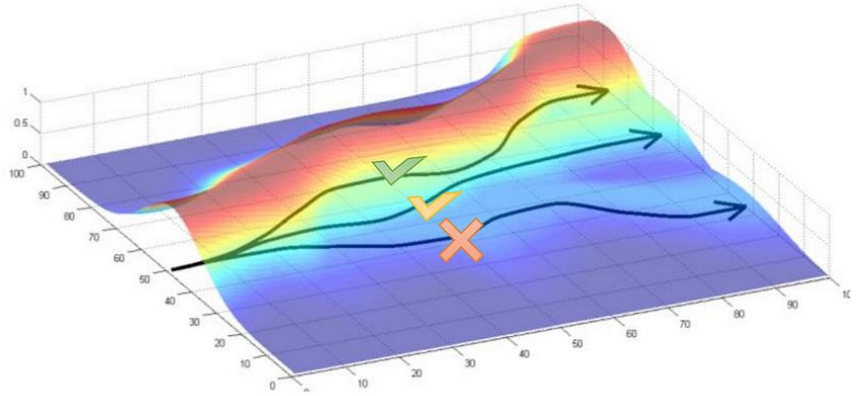
REINFORCE

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

- Gradient tries to:
  - **Increase the probability** of paths with **positive R**
  - **Decrease probability** of paths with **negative R**

- **Notice that**: Likelihood ratio changes probabilities of experienced paths, does not try to change the paths (←→ path derivatives)

- Cons: learning is slow and has **high variance**



REINFORCE with baseline

- Adding an arbitrary baseline function $b(s)$ we **subtract** from the cumulative reward:

$$\nabla_\theta J(\theta) = \mathbb{E}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b(s_t))]$$

- Update rule:

$$\theta_{t+1} = \theta_t + \alpha(G_t - b(S_t))\nabla \ln \pi(A_t|S_t)$$

- What can we use as baseline?

Continuous control with Gaussian policies

- Assume the values for actions are Gaussian distributed

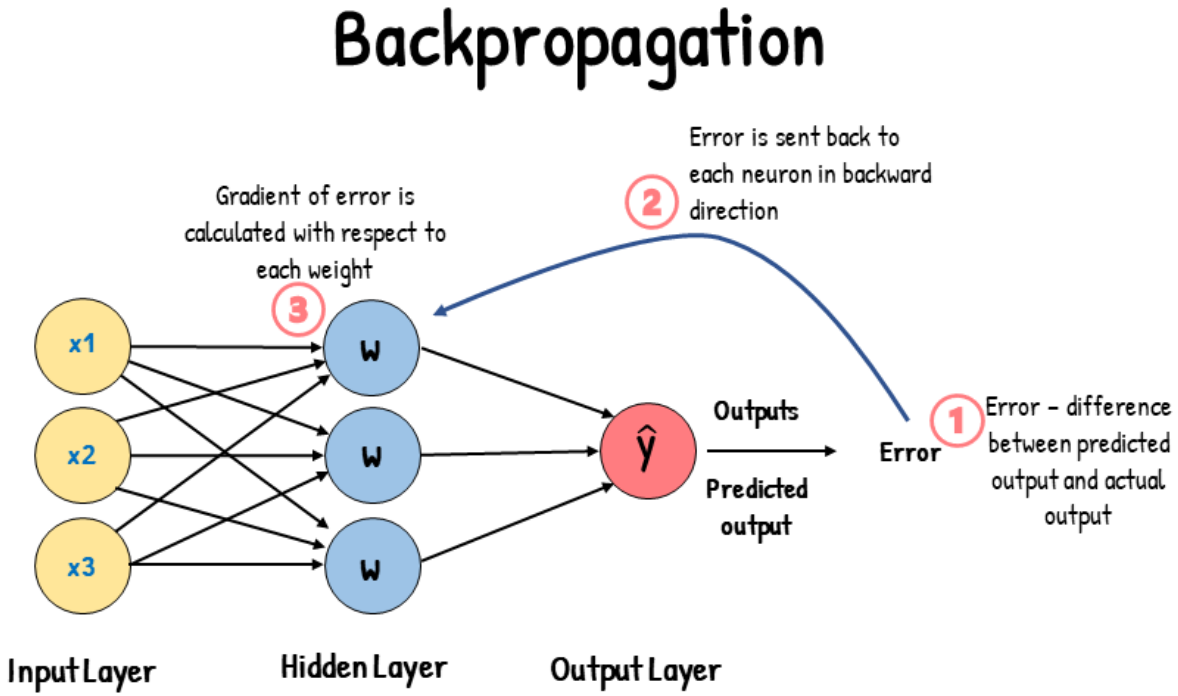
$$f(\boldsymbol{x}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\}$$

- Policy defined using a **Gaussian distribution** with **means computed from a deep neural network**

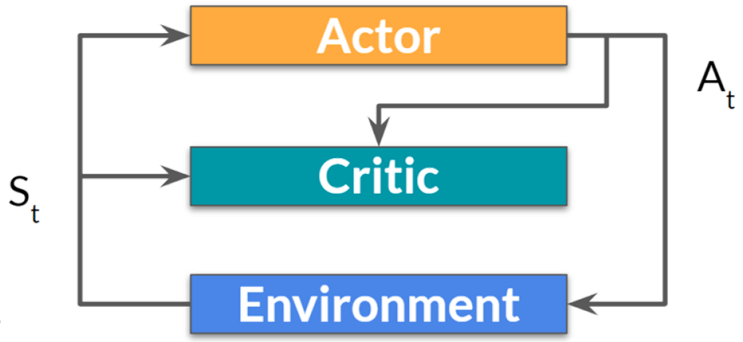
$$\pi_\theta(\mathbf{a}_t|\mathbf{s}_t) = \mathcal{N}(f_{\text{neural network}}(\mathbf{s}_t); \Sigma)$$

Training a Neural Network in PyTorch

- https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html#optimization-loop



Actor-Critic methods



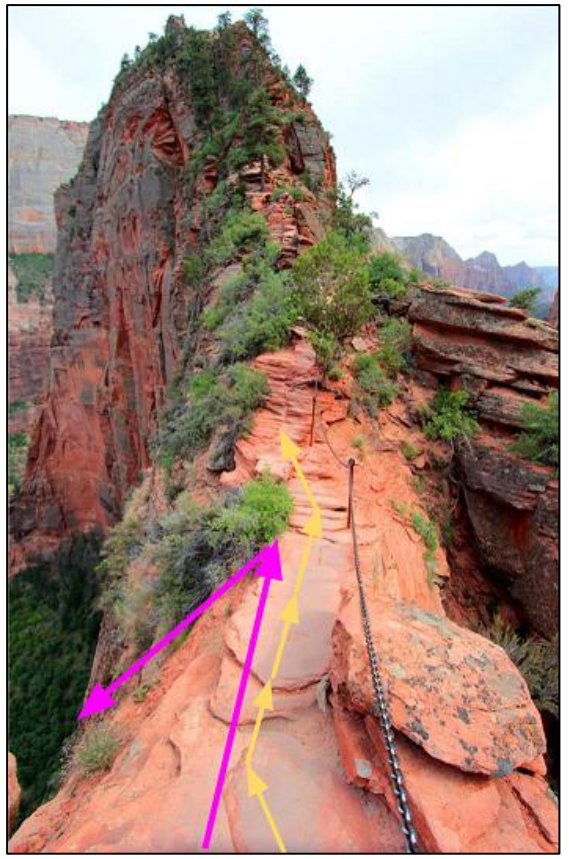
- Temporal Difference (TD) version of Policy Gradient
- **Hybrid architecture** combining Value-based and Policy-based methods that to **learn the value function in addition to the policy**
- It helps to stabilize the training by **reducing the variance** learning two function approximations:
  - An *Actor* that updates the policy distribution in the direction suggested by the *Critic* and **controls how our agent behaves** (Policy-Based method): $\pi_\theta(s)$
  - A *Critic* that estimates the value function and **measures how good the taken action is** (Value-Based method): $Q_w(s,a)$

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) Q_w(s_t,a_t)]$$

Also here we can use baselines! (e.g., A2C)

Proximal Policy Optimization (PPO)

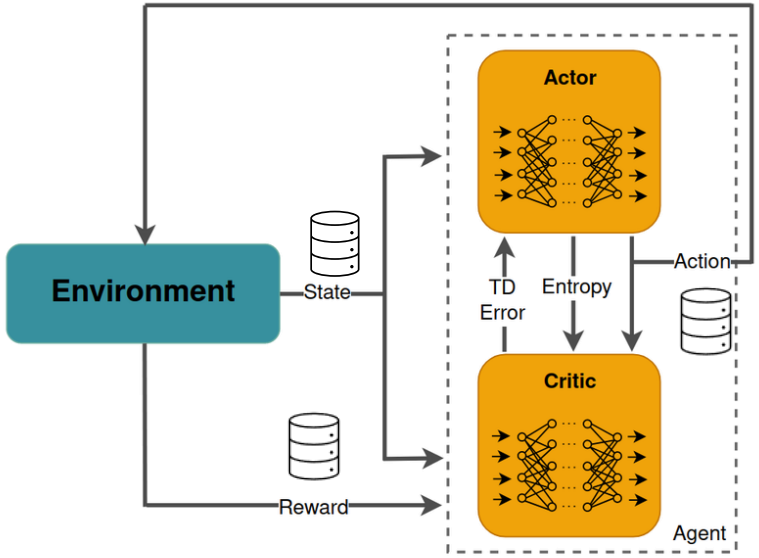
- **On-policy** algorithm
- It improves the agent's training stability by **avoiding policy updates that are too large → training is more stable**
- Probability ratio between old and new policies: $r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$
- Clip this ratio to a specific range: $[1-\epsilon, 1+\epsilon\,]$



Soft Actor-Critic (SAC)

- **Off-policy** algorithm (using a *replay buffer*)

- **Entropy regularization**: the policy is trained to **maximize a trade-off between expected return and entropy**, a measure of randomness in the policy
  - Close connection to the *exploration-exploitation* trade-off!
  - Prevent the policy from prematurely converging to a bad local optimum.



- The policy is trained with the objective to maximize the expected return and the entropy at the same time:

$$J(\theta) = \sum_{t=1}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_{\pi_\theta}}[r(s_t,a_t) + \alpha\mathcal{H}(\pi_\theta(.\,|s_t))]$$

Stable-Baselines3

```
pip install stable_baselines3==2.7.0
```

- [Library](#) of reliable implementations of **reinforcement learning algorithms**

Name	Box	Discrete	MultiDiscrete	MultiBinary	Multi Processing
ARS [1]	✓	✓	✗	✗	✓
A2C	✓	✓	✓	✓	✓
DDPG	✓	✗	✗	✗	✓
DQN	✗	✓	✗	✗	✓
HER	✓	✓	✗	✗	✓
PPO	✓	✓	✓	✓	✓
QR-DQN [1]	✗	✓	✗	✗	✓
RecurrentPPO [1]	✓	✓	✓	✓	✓
SAC	✓	✗	✗	✗	✓
TD3	✓	✗	✗	✗	✓
TQC [1]	✓	✗	✗	✗	✓
TRPO [1]	✓	✓	✓	✓	✓
Maskable PPO [1]	✗	✓	✓	✓	✓